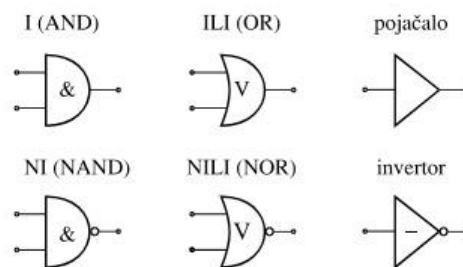


Pitanja 1. lab

- Algebra logike (struktura)
 - $\langle x, \&, V, -, =, S \rangle$
 - Booleova varijabla x
 - Vrijednosti 0 i 1 iz S
 - skup Booleovih konstanti $S = \langle 0, 1 \rangle$
 - Elementarni operatori
 - Konjunkcije ($\&$)
 - Disjunkcije (V)
 - Negacije ($-$)
 - Jednakost ($=$)

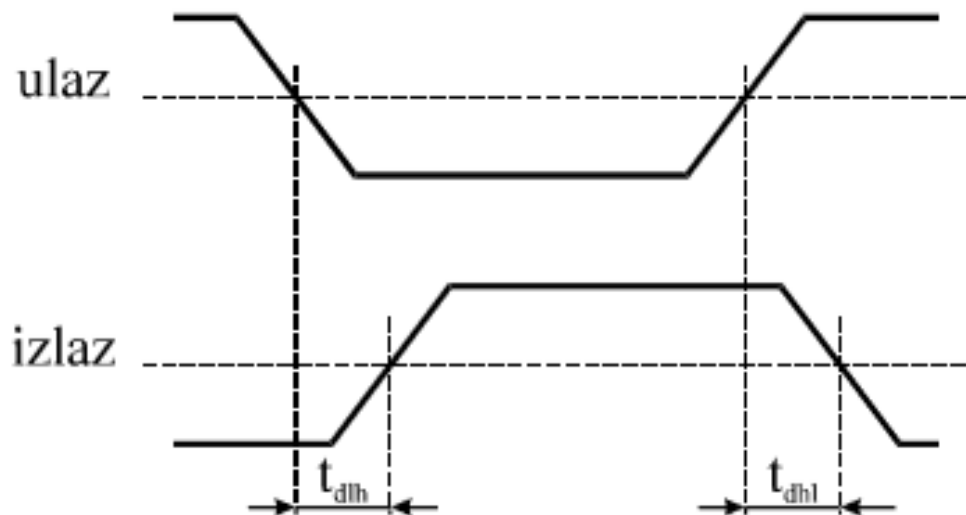
x_1	x_2	$x_1 \& x_2$	$x_1 \vee x_2$	$(x_1)^-$
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

- Elementarna logička vrata



- Podjela integriranih krugova po gustoći integracije
 - Po gustoći integracije vrsimo podjelu na SSI, MSI, LSI i VLSI integrirane krugove
 - SSI (niski stupanj integracije) realizira se integracijom do 12 ekvivalentnih logičkih vrata (oko 100) elemenata po pločici silicija
 - MSI (srednji stupanj integracije) podrazumijeva 100 vrata (1000 elemenata)
 - LSI (visoki stupanj integracije) podrazumijeva 1000 vrata (10 000 elemenata)
 - VLSI (vrlo visoki stupanj integracije) više od 10 000 vrata po pločici (100 000 elemenata)
- Faktor ulaznog i izlaznog grananja
 - Faktor ulaznog grananja kaže nam koliko stvarni ulaz opterećuje izlaz na koji je spojen u odnosu na standardni ulaz za promatranu tehnologiju i varijantu izrade. Ovaj faktor je najčešće 1. Ulaz u logičkoj 1 troši struju I_{ih} , a u logičkoj – daje struju I_{il} . Stoga je faktor grananja za određeni integrirani krug dat funkcijom minimuma: $FG = \min(I_{ohM} / I_{ih} ; I_{olM} / I_{il})$
 - Faktor izlaznog grananja (FAN-OUT) kaže nam koliko standardnih ulaza možemo spojiti na jedan izlaz, a da pri tom naponi signala ostanu u propisnim granicama. Za TTL i NMOS tehnologiju te su granice:
 - 0.8V max za logičku 0 (V_{ol})
 - 2.4V min za logičku 1 (V_{oh})

- Vrijeme kašnjenja
 - Definirano je za prijelaze na izlazu:
 - t_{dhl} = prijelaz iz 1 u 0
 - t_{dlh} = prijelaz iz 0 u 1
 - Oba vremena mjerimo kao kašnjenje između sredine brida ulaznog i sredine brida izlaznog signala logičkih vrata



- Mjera kvalitete
 - Produkt potrošnje i vremena kašnjenja koji ima dimenziju energije (J)

Pitanja 2. lab

- **Minterm**
 - Minterm i-tog retka od n varijabli je konjunkcija svih n varijabli, gdje varijable koje u pripadnoj kompleksiji imaju vrijednost 1 u konjunkciju ulaze nenegirane, a one ostale ulaze negirane. m_i je jednak jedinici kada uvrstimo pripadnu kompleksiju, a jednak nuli za sve ostale
- **PDNO**
 - Potpuni Disjunktivni Normalni Oblik je disjunktivna veza onih minterma za koje je vrijednost funkcije $T_i = 1$ (zbog ' $x \& 0 = 0$ ' i ' $x \vee 0 = x$ ')

$$f(x_1, x_2, \dots, x_n) = \bigvee_{i=0}^{2^n-1} (T_i \& m_i)$$

- **Tablica istine**
 - Osnovni način zapisivanja Booleove funkcije
 - S lijeve strane upisujemo sve kompleksije u prirodnom binarnom nizu, a s desne strane upisujemo vrijednosti jedne ili više funkcija (koje su funkcije istih varijabli)
 - Svakoј funkciji pripada jedan stupac desne strane tablice
 - Vrijednosti funkcije mogu biti iz skupa $\{0,1\}$, ili nedefinirane (R)
 - Svaki redak označavamo rednim brojem i od 0 do 2^n-1 , koji odgovara vrijednosti pripadne kompleksije varijabli promatrane kao prirodni binarni broj

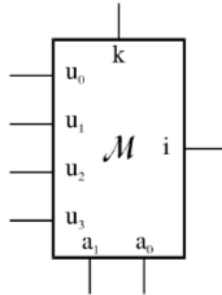
i	x_1	x_2	y_1	y_2	y_3
0	0	0	0	1	1
1	0	1	1	0	1
2	1	0	1	1	0
3	1	1	0	0	1

- **Zaokruživanje i ispisivanje površine (pravilo)**
 - Upisati funkciju u Veitchev dijagram i to samo 1 za PDNO, odnosno samo 0 za PKNO. U oba slučaja upisujemo r za redundantne kompleksije
 - Svakom mintermu za koji je funkcija jednaka jedinici tražimo susjeda. Ukoliko promatrani minterm nema susjednog člana, proglašavamo ga elementarnim članom
 - Površine susjednih minterma udružujemo u veće pravokutnike koji predstavljaju izraze dužine n-1, gdje je n broj varijabli. Eliminirana je ona varijabla po kojoj se dva udružena područja razlikuju. Članovima dužine n-1 ponovo tražimo susjede koji također moraju imati dužinu n-1. Ukoliko ih pronađemo, eliminiramo još jednu varijablu i dobijemo član dužine n-1. Članovi koji nemaju susjede, smatraju se elementarnima. Višestruko zaokruživanje neke površine ekvivalentno je dopisivanju postojećeg člana normalnog oblika. Zaokruživanje redundantnog člana R ekvivalentno je njegovom dopisivanju u algebarski oblik
 - Postupak ponavljamo sve dok ne ostanu samo elementarni članovi. Cilj je sve jedinice obuhvatiti što manjim brojem što većih površina
 - Ispisujemo minimalni algebarski oblik funkcije (MDNO ili MKNO), tako da za svaku površinu ispišemo pripadni elementarni član. Uzimamo samo nužne elementarne članove

Pitanja 4. lab

- Multiplekser (def, slika, formule, primjer)
 - Logički sklop koji na informacijski izlaz i propušta vrijednost onog od $n=2^m$ informacijskih ulaza $u_0...u_{n-1}$, čiji je redni broj prisutan u prirodnom binarnom obliku na m adresnih ulaza $a_0...a_{m-1}$

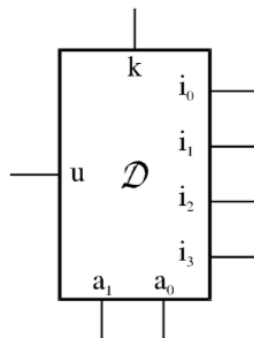
$$i = \bigvee_{j=0}^{2^m-1} m_j \& u_j$$



Slika 4.1. - Multiplekser $m=2$

- Demultiplekser (def, slika, formule, primjer)
 - Logički sklop koji vrijednost informacijskog ulaza u propušta na onaj od $n=2^m$ informacijskih izlaza $i_0...i_{n-1}$, čiji je redni broj prisutan u prirodnom binarnom obliku na m adresnih ulaza $a_0...a_{m-1}$

$$i_j = m_j \& u$$

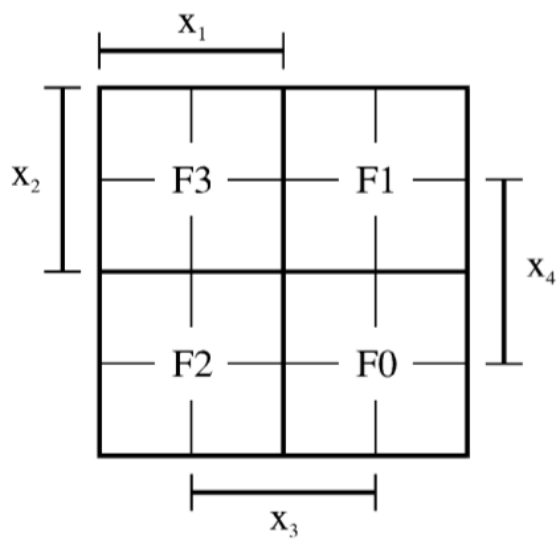


Slika 4.2. - Demultiplekser $m=2$

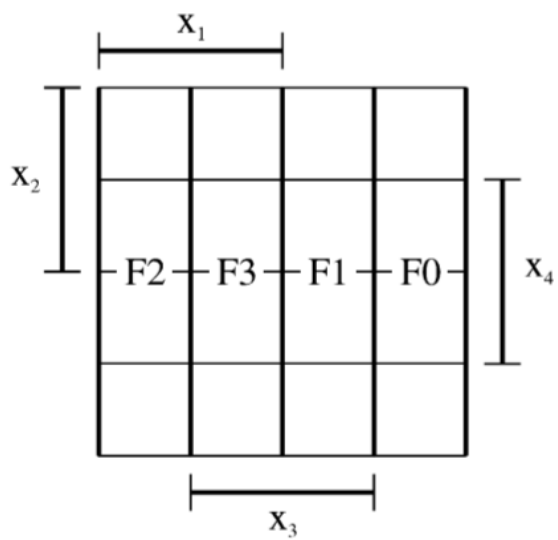
- Realizacija B.F. MUXOM i DEMUXOM za $m=n$ (primjeri bez formula)
- Raspodjela na preostale funkcije korištenjem V.D.-a (bez formule)

- Izborom jedne varijable, PDNO funkcije i Veitchev dijagram se raspadaju na dva dijela. Svaki dio predstavlja po jednu preostalu funkciju. Izborom sljedećih adresnih varijabli, PDNO i Veitchev dijagram se raspadaju na 4, 8, itd. dijelova. Nakon upisivanja funkcije u Veitchev dijagram, postupak izbora adresnih varijabli obavljamo u sljedećim koracima:
 - Sagledamo mogućnost minimizacije i za adresnu varijablu izaberemo onu po kojoj je najteže vrišti minimizaciju
 - Podijelimo Veitchevog dijagram na dva dijela s obzirom na izabranu adresnu varijablu, te za cijeli dijagram ponavljamo prethodni postupak, tako da e sljedeća izabrana varijabla zajednička za sva područja
 - U okviru dobivenih područja vrsimo minimizaciju preostalih funkcija koje realiziramo logičkim vratima i dovodimo na odgovarajuće ulaze multipleksera
- Za funkciju sa četiri varijable moguće je kod realizacije sa multiplekserom sa dva adresna ulaza ($m=2$) ostvariti šest različitih kombinacija adresnih ulaza, za koje podjela Veitchevog dijagrama na područja izgleda:

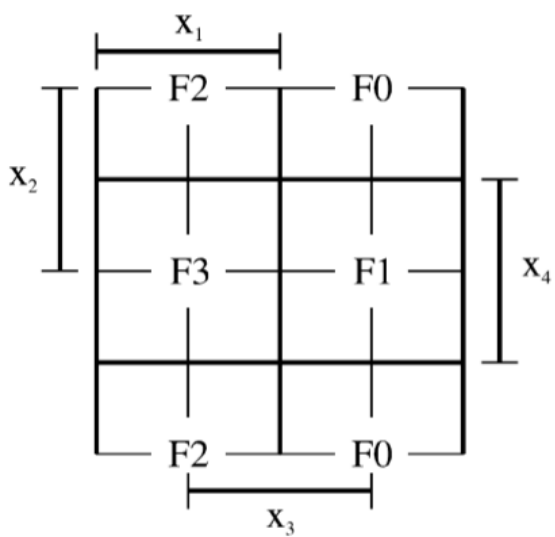
podjela po x_1 i x_2



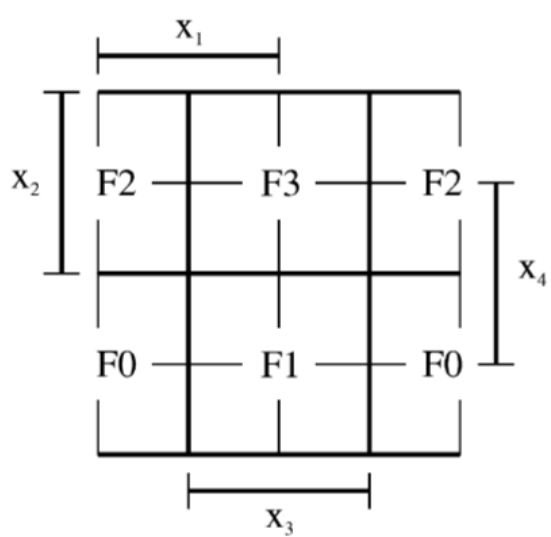
podjela po x_1 i x_3



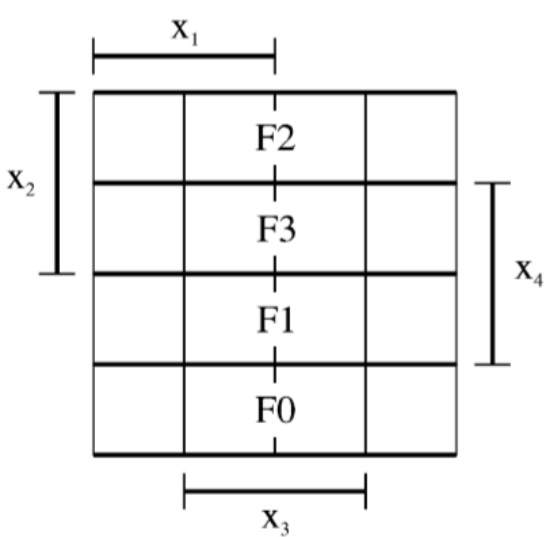
podjela po x_1 i x_4



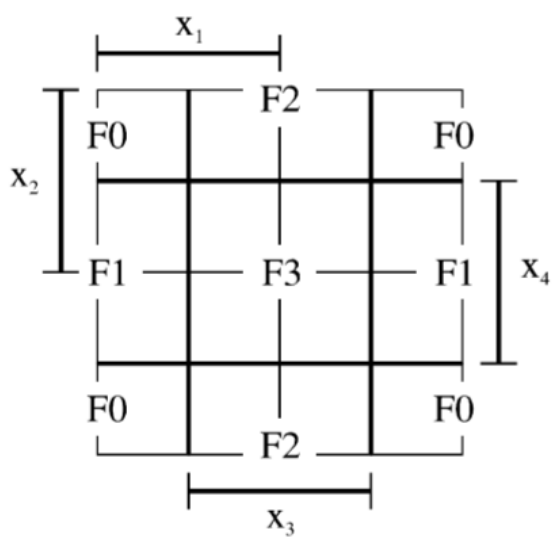
podjela po x_2 i x_3



podjela po x_2 i x_4

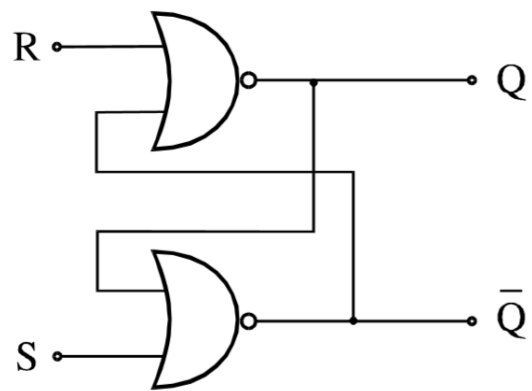
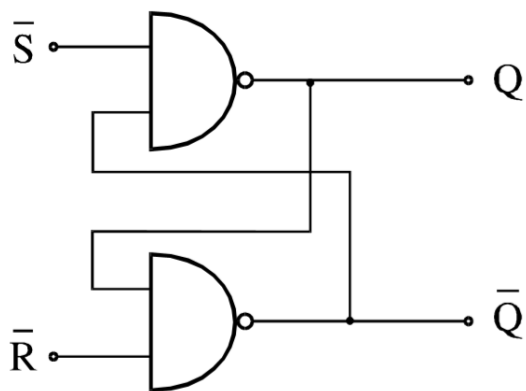


podjela po x_3 i x_4

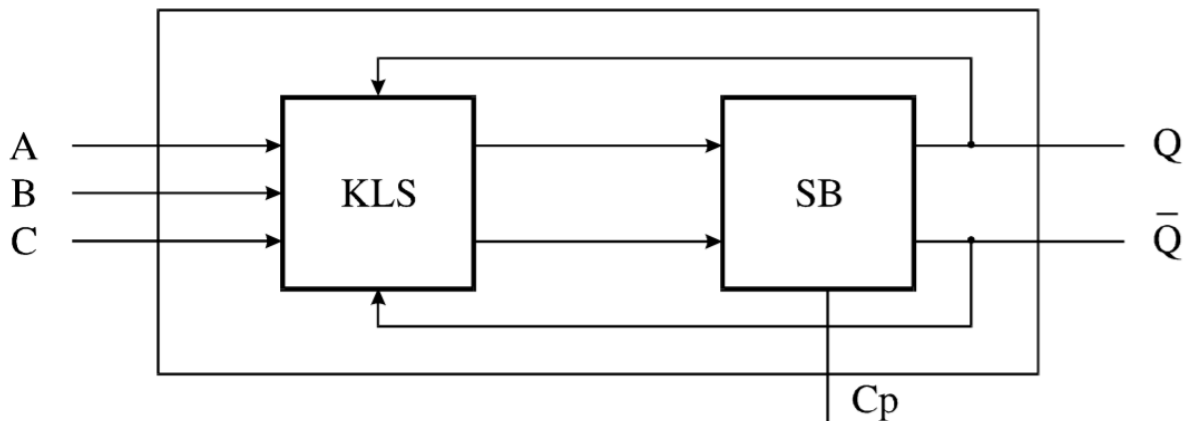


Pitanja 6. lab

- Bistabil (def, slika, primjer)
 - Osnovni memorijski element za pamćenje vrijednosti Booleove varijable, ima dva stabilna unutrašnja stanja (0 ili 1)
 - Promjena stanja memorijskog elementa ovisi o trenutnoj vrijednosti na njegovim ulazima $q_1, q_2, \dots, q_n$, kao i o njegovom unutrašnjem stanju
 - Unutrašnje stanje očitavamo na izlazima Q i $\bar{Q}$
 - Simetričan sklop i kod realizacije TTL tehnologijom sastoji se od dva tranzistora, koji sačinjavaju dvostruko pojačalo s jakom pozitivnom povratnom vezom
 - Veliko pojačanje u petlji povratne veze ne dozvoljava da sklop oscilira, nego dovodi do toga da jedan tranzistor vodi dok drugi ne vodi
 - Bistabil ostaje u istom stanju sve dok se na ulazu ne pojavi aktivni impuls ili nivo koji ga prebacuje u drugo

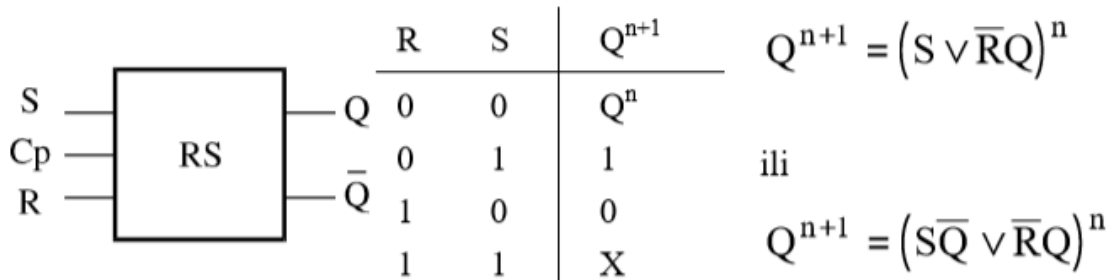


- Opći Bistabil (slika)



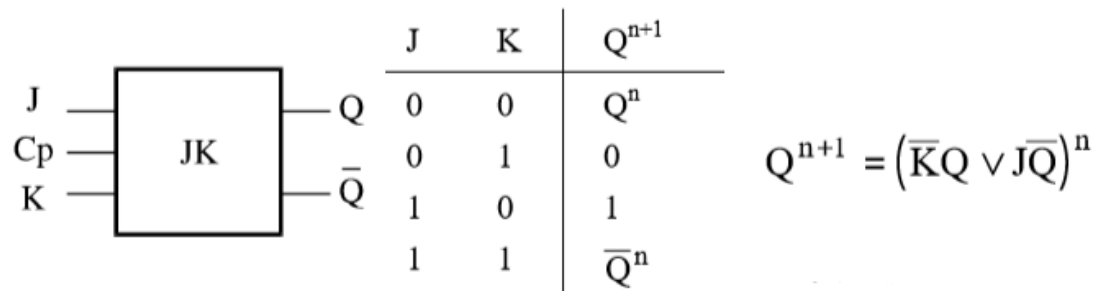
- Standardni bistabil

- RS, JK, D i T bistabili
- Njihove funkcije prijelaza zovu se karakteristične jednađbe
- Realiziraju se kao sinkroni (imaju taktni ulaz), a često im se izvode i asinkroni R i S ulazi radi dovođenja u poznato početno stanje
- **RS bistabil**
 - elementarni bistabil, ali mu je prijelaz neodređen kada su oba ulaza u 1
 - uvjet ispravne primjene je $RS = 0$



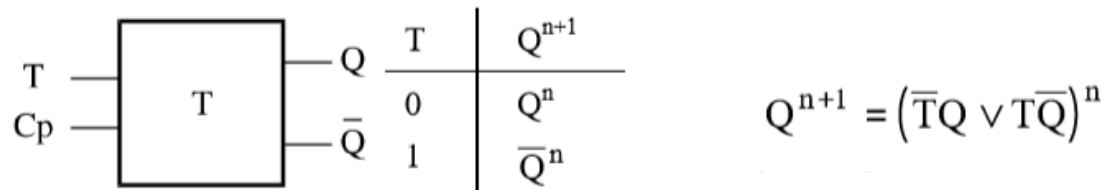
- **JK bistabil**

- Univerzalni bistabil koji realizira sve vrste preslikavanja



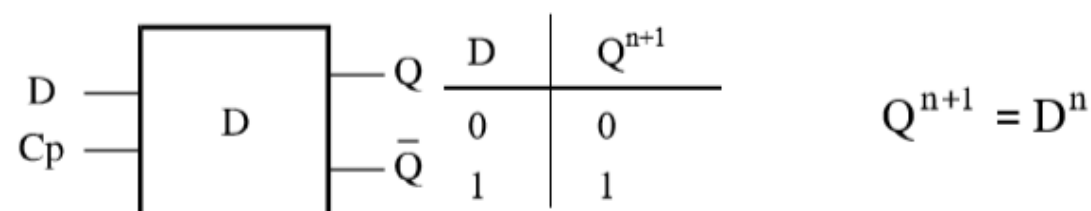
- **T bistabil**

- Mijenja stanje nastupom taktnog signala (uz $T=1$), pa ga koristimo za dijeljenje frekvencije sa 2 i realizaciju brojala



- **D bistabil**

- Ponaša se kao sklop za kašnjenje, tj. vrijednost ulaza D se prenese na izlaz nakon nastupa taktnog signala
- Kašnjenje je jednako periodu taktnog signala

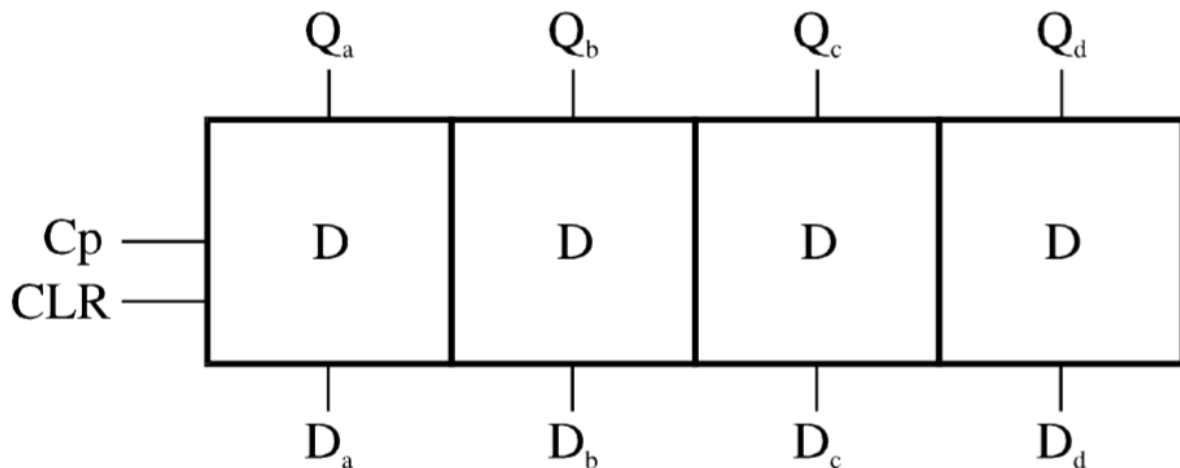


- Brzina rada ovisna je o kašnjenju između aktivnog dijela taktnog signala i promjene na izlazu bistabila, ovo kašnjenje ne treba miješati sa funkcijom bistabila kao memorijskog elementa, odnosno elementa za kašnjenje, koje se odvija pod kontrolom taktnog signala
- Skraćena i potpuna tablica prijelaza s primjerom i funkcije prijelaza

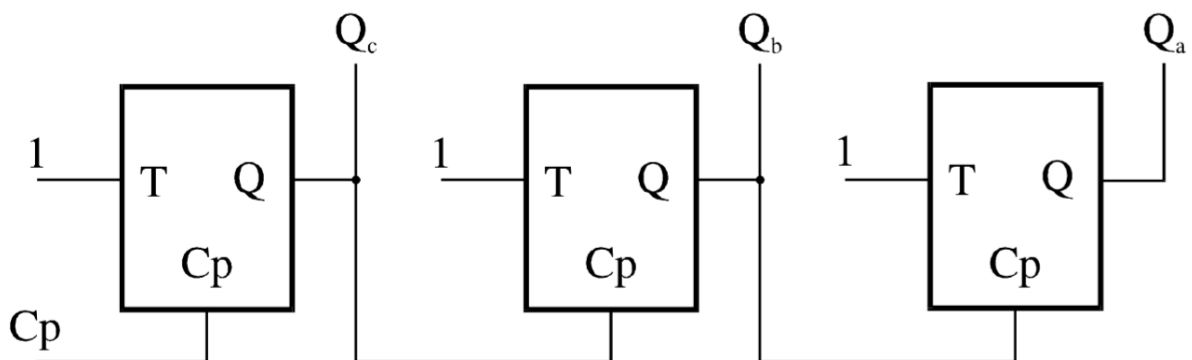
potpuni oblik						skraćeni oblik					
$(q_1,$	$q_2,$	$\dots,$	$q_n,$	$Q)^n$	Q^{n+1}	$(q_1,$	$q_2,$	$\dots,$	$q_n)^n$	Q^{n+1}	
0	0	...	0	0	1	0	0	...	0	$\overline{Q}^n$	
0	0	...	0	1	0	0	0	...	1	Q^n	

$$Q^{n+1} = f(Q, q_1, \dots, q_n)^n \quad \text{ili} \quad Q^{n+1} = (G_1 Q \vee G_2 \overline{Q})^n$$

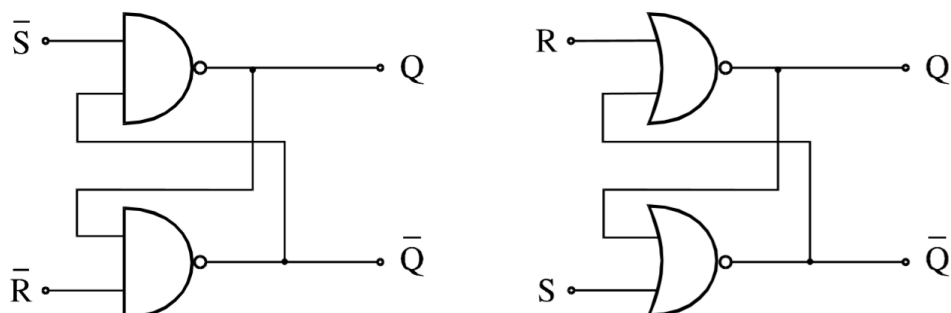
- Registar (slika, def)
 - Sklop koji nam služi za pamćenje kodnih rijeci dužine m bitova
 - Za pamćenje svakog pojedinog bita koristimo vlastiti D bistabil, čiji je taktni ulaz spojen na zajednički taktni signal Cp
 - Ulazi u bistabile D su ulazi, registra, a izlazi bistabil Q su izlazi registra
 - Kako se pamćenje podataka vrši u istom trenutku, sinkrono sa taktnim signalom, govorimo o paralelnim ulazima i izlazima
 - Registri se izvode sa 4 i 8 bistabil i najčešće raspolažu sa asinkronim CLR (R) ulazom, kojim se dovode u početno stanje 0



- Brojilo (slika, def)
 - Sklop tako povezanih m bistabil, da njihovo sljedeće stanje ovisi samo o prethodnom
 - Prijelaz nastaje u trenutku nastupa taktnog signala
 - brojilo je jedna vrsta generatora sekvence, dakle automata bez vanjskih ulaza, kod kojega su stanja ujedno i izlazi, i zbog toga nije moguće ponavljanje simbola u izlaznoj sekvenci
 - prolaskom kroz sva stanja brojilo na izlazima generira konačnu sekvencu kodnih kompleksija
 - ukoliko je to prirodni binarni niz od 2^m kodnih kompleksija, govorimo o binarnom brojilu, a ako su to binarno kodirane dekadске znamenke (BCD kod), govorimo o dekadskom brojilu



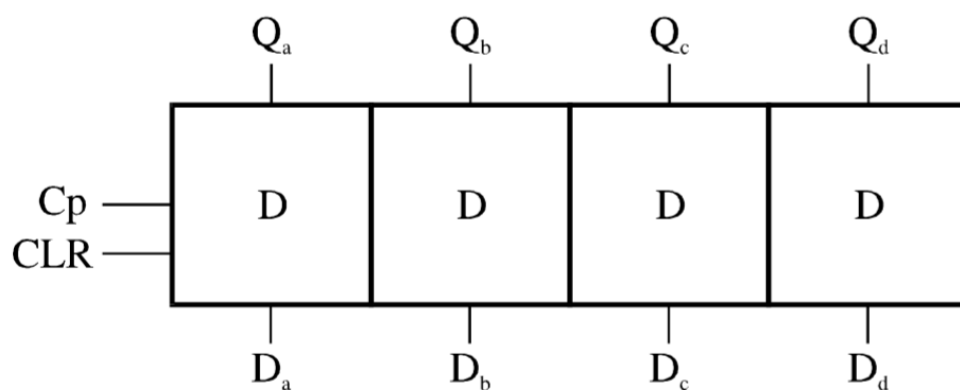
1. Sto je bistabil
 - Bistabil je osnovni memorijski element za pamćenje vrijednosti Booleove varijable, ima dva stabilna unutrašnja stanja (0 i 1)
2. Kako zadajemo bistabil
 - Bistabile zadajemo tablicama prijelaza i funkcijama prijelaza
3. Osnovna shema bistabil



4. Skraćena tablica

potpuni oblik						skraćeni oblik					
$(q_1, q_2, \dots, q_n, Q)^n$						$(q_1, q_2, \dots, q_n)^n$					
0	0	...	0	0	Q^{n+1}	0	0	...	0	Q^{n+1}	$\bar{Q}^n$
0	0	...	0	1	0	0	0	...	1	Q^n	

5. Registar je sklop koji nam služi za pamćenje kodnih rijeci dužine m bitova. Za pamćenje svakog pojedinog bita koristimo vlastiti D bistabil, čiji je taktni ulaz spojen na zajednički taktni signal Cp. Ulazi u bistabile D su ulazi registra, a izlazi bistabil Q su izlazi registra



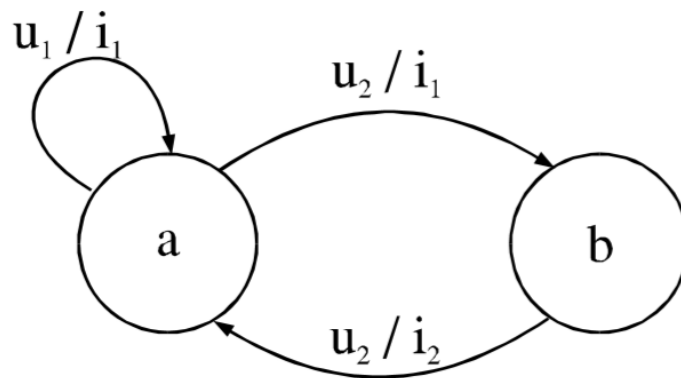
Pitanja 7./8. lab

- Automat (def.)
 - Logička struktura čije ponašanje ovisi o trenutnoj vrijednosti ulaza (ako postoje) i o događajima iz prethodnih diskretnih vremenskih trenutaka
 - To znaci da automat ima neku vrstu memorije, kojom pamti prethodne događaje
 - Stanje memorije određuje sekvencu proteklih događaja i zajedno sa trenutnim ulazima definira prijelaz u sljedeće stanje
 - Govorimo o automatima koji su:
 - Digitalni – ulazi i izlazi su binarne numeričke vrijednosti
 - Diskretni – rade u diskretnom vremenu pod kontrolom taktnog signala
 - Konačni – broj stanja memorije je konačan i time je automat ostvariv
 - Determinirani – funkcije prijelaza i izlaza su jednoznačne
 - Potpuno ili nepotpuno specificirani – sekvencu ulaznih nema, odnosno ima unutrašnja ograničenja
 - Sinkroni – rad bistabil je sinkroniziran taktnim signalom
- Model realizacije automata

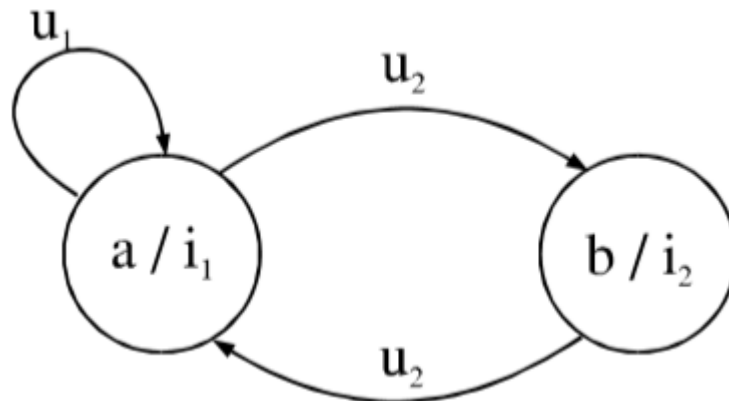


- Generator sekvence (def.)
 - Automat bez ulaza, koji na svom izlazu daje konačnu sekvencu izlaznih slova
 - S obzirom na način rada, razlikujemo generatore koji konačnu sekvencu ponavljaju beskonačan broj puta, te one koji se nakon prolaza kroz sva stanja zaustavljaju
- Brojilo (def. Automat)
 - Ukoliko se slova sekvence ne ponavljaju, možemo stanja automata i izlazna slova kodirati identično, takav generator sekvence nazivamo brojilom
- Zadavanje automata
 - Kod početnog zadavanja automata, ideju o funkciji automata moramo zabilježiti nekim od standardnih jezika
 - Sekvencu slova ulaznog alfabeta koja se u vremenskom slijedu pojavljuje na ulazima automata, nazivamo ulazna riječ
 - Analogno, sekvencu izlaznih slova čini izlaznu riječ
 - Automat na osnovu ulazne riječi generira izlaznu

- Zapisivanje automata (primjeri grafova i tablice)
 - Automat zapisujemo tako da definiramo skupove U , I i S , te zapišemo funkcije δ i λ
 - U praksi je dovoljno zapisati funkcije, jer tako implicitno zadajemo i skupove
 - Formalne jezike kojima zapisujemo funkcije nazivamo standardnim jezicima
 - Automate zapisuje grafički – pomoću orijentiranih grafova i tabelarno – pomoću tablica prijelaza i izlaza
 - Zapisivanje automata pomoću orijentiranog grafa:
 - Grafove formiramo od krugova koji predstavljaju stanja automata povezanih usmjerenim duzinama koje označavaju prijelaze iz stanja u stanje
 - Kod Mealyeva modela krugovima su označena samo unutrašnja stanja, a dužine predstavljaju prijelaze i njima dodijeljene izlazne simbole



- Kod Mooreova modela krugovima su osim stanja označeni i izlazi kao funkcije stanja, a usmjerene dužine predstavljaju samo prijelaze



- Zapisivanje automata pomocu tablice prijelaza i izlaza
 - Automat zadajemo tablicom tako da funkciju prijelaza upisujemo u tablicu prijelaza, a funkciju izlaza u tablicu izlaza
 - Retke tablice dodijeljimo stanjima $s(t)$, a stupce ulaznim slovima $u(t)$
 - Presjkeciste stupca i retka odgovara clanu karezijevog pridukta $S \times U$, te na to mjesto upisjuemo $s(t+1)$ kod tablice prijelaza, odnosno $i(t)$ kod tablice izlaza
 - Uobicajeno je tablice prijelaza i izlaza pisati zajedno
 - Tablica prijelaza i izlaza za Mealyev model:

	$u_1(t)$	...	$u_l(t)$	$u_1(t)$	...	$u_l(t)$
$s_1(t)$	$s_i(t+1)$	...	$s_i(t+1)$	$i_i(t)$	...	$i_i(t)$
.	.		.	.		.
.	.		.	.		.
.	.		.	.		.
$s_k(t)$	$s_i(t+1)$	...	$s_i(t+1)$	$i_i(t)$	...	$i_i(t)$

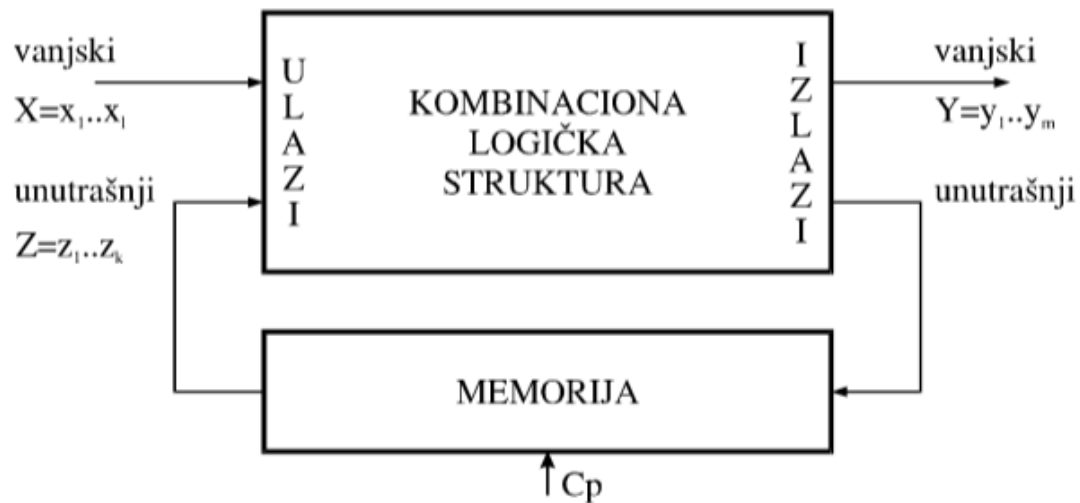
- Za Moorev model tablica izlaza ima samo jedan stupac jer izlazi ovise isključivo o $s(t)$:

	$u_1(t)$	...	$u_l(t)$	izlaz
$s_1(t)$	$s_i(t+1)$	...	$s_i(t+1)$	$i_i(t)$
.	.		.	.
.	.		.	.
.	.		.	.
$s_k(t)$	$s_i(t+1)$	...	$s_i(t+1)$	$i_i(t)$

- Formalni opis automata (petorka)
 - $A = \langle U, S, I, \delta, \lambda \rangle$
 - $U = (u_1, u_2, \dots, u_p)$ kodiran s $X = (x_1, x_2, \dots, x_l)$, $2^l \geq p$
 - $I = (i_1, i_2, \dots, i_q)$ kodiran s $Y = (y_1, y_2, \dots, y_m)$, $2^m \geq q$
 - $S = (s_0, s_1, \dots, s_n)$ kodiran s $Z = (z_1, z_2, \dots, z_k)$, $2^k \geq n$
 - Skup u nazivamo ulaznim alfabetom automata, skup I predstavlja izlazni alfabet, a skup S je skup unutrašnjih stanja automata; ukoliko je ovaj skup konačan govorimo o konačnom automatu
 - Skup X je skup ulaznih, a skup Y izlaznih varijabli
 - Skup z je skup bistabila, odnosno njihovih izlaznih varijabli
 - Funkcije δ i λ opisuju prijelaz i izlaz automata

- 1. i 2. korak strukturne sinteze
 - U prvom koraku strukturne sinteze vrsimo kodiranje automata
 - Određujemo broj bistabil memorije i broj ulaznih i izlaznih varijabli, te vrsimo dodjeljivanje kodnih kompleksija stanjima i simbolima, pri tome trebaju bit zadovoljeni uvjeti kodiranja:
 - Skup U sa p članova u kodiramo kompleksijama od l varijabli x: $2^l \geq p$
 - Skup I sa q članova i kodiramo kompleksijama od m varijabli y: $2^m \geq q$
 - Skup S sa n članova s kodiramo kompleksijama od k bistabil z: $2^k \geq n$
 - Način kodiranja bitno utječe na veličinu automata, kod kodiranja ulaza i izlaza moramo voditi računa o kompatibilnosti sa izvoristom i odredištem informacije i najčešće nemamo potpunu slobodu rada
 - Kodiranje stanja realiziramo dodjeljivanjem kompleksija stanja bistabil stanju automata, ne postoje egzaktne metode kodiranja koje garantiraju minimalni automat, ali kao uspješnu strategiju možemo koristiti pravilo da stanjima među kojima imamo veće mogućnosti prijelaza dodjeljujemo susjedne kompleksije ili barem one sa sto manjom distancom
 - U postupku kodiranja koristimo Veitchjev dijagram, jer su susjedne kompleksije na njemu i fizički susjedne
 - Početno stanje u pravilu kodiramo kompleksijom 00...0
 - Drugi korak strukturne sinteze je uvrštavanjem kodnih kompleksija u tablicu prijelaza i izlaza automata; time dobijemo novi tablicu, čije dijelove možemo prepoznati kao tablice prijelaza za opće bistabile i tablice istine za izlazne varijable; na osnovu tih tablica vrsimo minimizaciju i realizaciju Booleovih funkcija i općih bistabil
 - U slučaju kad se radi o generatoru sekvence koji nema ulaze, postupak je identičan postupku za automat, osim sto su tablice jednostavnije jer izostavljamo stupce za ulaze
- Mealyev i Mooreov model
 - Mealy: $i(t) = \lambda(s(t), u(t))$; $i(t) = \lambda(SxU, t)$ $SxU \rightarrow I$
 - Izlazni simbol u trenutku n ovisi o trenutno prisutnom ulaznom simbolu, te trenutnom stanju, stoga reagira jedan period taktnog signala prije Mooreova, ali je nužno pojavu ulaznih simbola sinkronizirati sa taktnim signalom
 - Funkcija izlaza vrsi preslikavanje iz SxU u I
 - Moore: $i(t) = \lambda(s(t))$; $i(t) = \lambda(S, t)$ $S \rightarrow I$
 - Izlazni simbol je isključivo funkcija trenutnog stanja
 - Ulaz djeluje posredno preko sljedećeg stanja, pa ovaj automat zapravo uzrokuje ulaznu sekvencu
 - Funkcija izlaza vrsi preslikavanje iz S u I

1. Automat (def.)
 - Automat je logička struktura čije ponašanje ovisi o trenutnoj vrijednosti ulaza (ako postoje) i o događajima iz prethodnih diskretnih vremenskih trenutaka
2. Model realizacije automata (slika)



3. Formalni opis automata (formula i značenje znakova u toj formuli)
 - $A = \langle U, S, I, \delta, \lambda \rangle$
 - U – ulazni alfabet automata
 - $(u_1, u_2, \dots, u_p)$ kodiran s $X = (x_1, x_2, \dots, x_l)$, $2^l \geq p$
 - I – izlazni alfabet
 - $(i_1, i_2, \dots, i_q)$ kodiran s $Y = (y_1, y_2, \dots, y_m)$, $2^m \geq q$
 - S – skup unutrašnjih stanja
 - $(s_0, s_1, \dots, s_n)$ kodiran s $Z = (z_1, z_2, \dots, z_k)$, $2^k \geq n$
 - δ – funkcija prijelaza
 - λ – funkcija izlaza
4. Razlika između Mealy i Moore modela
 - Mealy: $i(t) = \lambda(s(t), u(t))$; $i(t) = \lambda(SxU, t)$ $SxU \rightarrow I$
 - Izlazni simbol u trenutku n ovisi o trenutno prisutnom ulaznom simbolu, te trenutnom stanju, stoga reagira jedan period taktnog signala prije Mooreova, ali je nužno pojavu ulaznih simbola sinkronizirati sa taktnim signalom
 - Funkcija izlaza vrši preslikavanje iz SxU u I
 - Moore: $i(t) = \lambda(s(t))$; $i(t) = \lambda(S, t)$ $S \rightarrow I$
 - Izlazni simbol je isključivo funkcija trenutnog stanja
 - Ulaz djeluje posredno preko sljedećeg stanja, pa ovaj automat zapravo uzrokuje ulaznu sekvencu
 - Funkcija izlaza vrši preslikavanje iz S u I

5. Prvi korak strukturne sinteze

- U prvom koraku strukturne sinteze vrsimo kodiranje automata
 - Određujemo broj bistabil memorije i broj ulaznih i izlaznih varijabli, te vrsimo dodjeljivanje kodnih kompleksija stanjima i simbolima, pri tome trebaju biti zadovoljeni uvjeti kodiranja:
 - Skup U sa p članova u kodiramo kompleksijama od l varijabli x: $2^l \geq p$
 - Skup I sa q članova i kodiramo kompleksijama od m varijabli y: $2^m \geq q$
 - Skup S sa n članova s kodiramo kompleksijama od k bistabil z: $2^k \geq n$
 - Način kodiranja bitno utječe na veličinu automata, kod kodiranja ulaza i izlaza moramo voditi računa o kompatibilnosti sa izvorištem i odredištem informacije i najčešće nemamo potpunu slobodu rada
 - Kodiranje stanja realiziramo dodjeljivanjem kompleksija stanja bistabil stanju automata, ne postoje egzaktno metode kodiranja koje garantiraju minimalni automat, ali kao uspješnu strategiju možemo koristiti pravilo da stanjima među kojima imamo veće mogućnosti prijelaza dodjeljujemo susjedne kompleksije ili barem one sa sto manjom distancom
 - U postupku kodiranja koristimo Veitchev dijagram, jer su susjedne kompleksije na njemu i fizički susjedne
 - Početno stanje u pravilu kodiramo kompleksijom 00...0